МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №6

з курсу: „Системи баз даних”

виконав студент 3-го курсу
групи ДА-11
Сидько Н.С.

КИЇВ
2023

Програмування реляційних СБД

Встановлення MySQL

Для встановлення some-mysql, пропишемо нижче наведену команду в Windows PowerShell

```
PS C:\Users\user> docker run --name some-mysql -e MYSQL_ROOT_PASSWORD=root-pw -d mysql
Unable to find image 'mysql:latest' locally
[2023-11-26T19:02:36.165436700Z][docker-credential-desktop.system][W] Windows version might
m cannot find the file specified.
latest: Pulling from library/mysql
8e0176adc18c: Pull complete
2d2c52718f65: Pull complete
d88d03ce139b: Pull complete
4a7d7f11aa1e: Pull complete
ce5949193e4c: Pull complete
f7f024dfb329: Pull complete
5fc3c840facc: Pull complete
509068e49488: Pull complete
cbc847bab598: Pull complete
942bef62a146: Pull complete
Digest: sha256:1773f3c7aa9522f0014d0ad2bbdaf597ea3b1643c64c8ccc2123c64afd8b82b1
Status: Downloaded newer image for mysql:latest
1788b6188268dc7cefa3573866fc35e82bcbf1fbbc453b56c43dc5bc8f56fa78
PS C:\Users\user>
```

Тоді запустимо Docker і бачимо тільки що встановлений some-mysql.

Name	Image	Status	CPU (%)	Port(s)	Last started
some-neo4j 848b790b1999	neo4j	Exited	0%	7474:7474 Show all ports (2	10 minutes ago
some-couchdb 5f1329f9c441	couchdb	Exited	0%	5984:5984	4 days ago
some-hbase 10e27fcb7f20	harisekhon/hbase	Exited (137)	0%		1 month ago
some-redis 86902c21c8c9	redis	Exited	0%	6379:6379	7 hours ago
some-postgres dfd4769d51d1	postgres	Exited	0%		1 month ago
some-mysql 1788b6188268	mysql	Running	0.73%		32 seconds ago

Пропишемо команду у Windows PowerShell: docker run –name phpMyAdmin -d –link some-mysql:db -p 8080:80 phpmyadmin

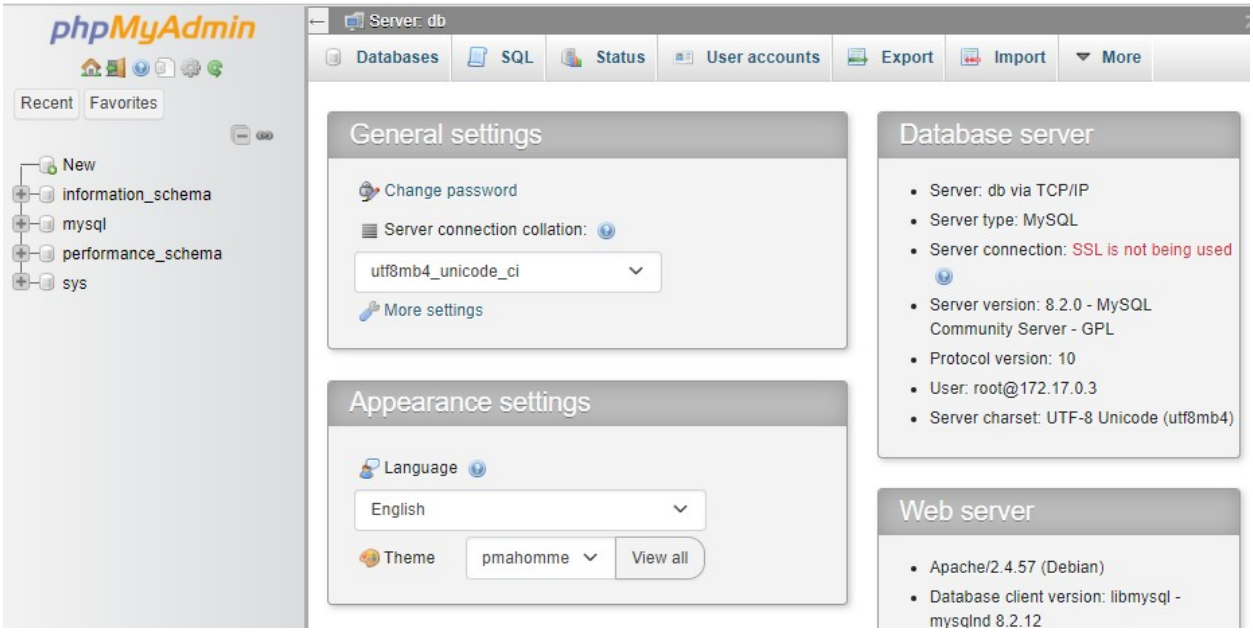
Тоді маємо:

Name	Image	Status	CPU (%)	Port(s)	Last started	Actions
some-neo4j 848b790b1999	neo4j	Exited	0%	7474:7474 Show all ports (2	17 minutes ago	
some-couchdb 5f1329f9c441	couchdb	Exited	0%	5984:5984	4 days ago	
some-hbase 10e27fcb7f20	harisekhon/hbase	Exited (137)	0%		1 month ago	
some-redis 86902c21c8c9	redis	Exited	0%	6379:6379	8 hours ago	
some-postgres dfd4769d51d1	postgres	Exited	0%		1 month ago	
some-mysql 1788b6188268	mysql	Running	0.46%		7 minutes ago	
phpmyadmin ec234b554d21	phpmyadmin	Running	0%	8080:80	19 seconds ago	

Перейдемо за посиланням localhost:8080. Бачимо веб інтерфейс сайту та проходимо реєстрацію

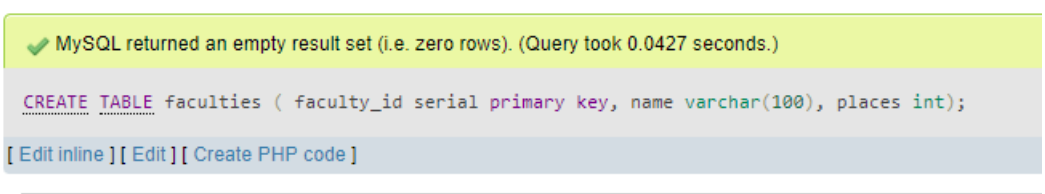


Тоді маємо наступну сторінку:



Приклади процедур в СБД

Дозапишемо таблицю faculties виконуючи запит **CREATE TABLE**



Бачимо вміст полів таблиці faculties:

Your SQL query has been executed successfully.

```
DESC faculties;
```

[Edit inline] [Edit] [Create PHP code]

Extra options

Field	Type	Null	Key	Default	Extra
faculty_id	bigint unsigned	NO	PRI	*NULL*	auto_increment
name	varchar(100)	YES		*NULL*	
places	int	YES		*NULL*	

Дозапишимо нові екземпляри даних використовуючи **INSERT INTO**

5 rows inserted.
Inserted row id: 5 (Query took 0.0176 seconds.)

```
INSERT INTO faculties (name, places) VALUES ('Math', 300), ('History', 200),('Law', 100),('Economy', 50),('Biology', 25);
```

[Edit inline] [Edit] [Create PHP code]

Дозапис procedure без параметрів

Ствремо procedure котра зчитує дані таблиці faculties:

```
1 DELIMITER //
2
3 CREATE PROCEDURE fetchData()
4 BEGIN
5   SELECT * FROM faculties;
6 END //
7
8 DELIMITER ;
```

Викличемо функцію **fetchData()** :

Showing rows 0 - 4 (5 total, Query took 0.0006 seconds.)

```
CALL fetchData();
```

Edit inline] [Edit] [Create PHP code]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

faculty_id	name	places
1	Math	300
2	History	200
3	Law	100
4	Economy	50
5	Biology	25

Розмежувач запитів і операторів в MySQL

В MySQL для відокремлення запитів зчиування / дозапису по умові використовується розмежувач “ **;** ”

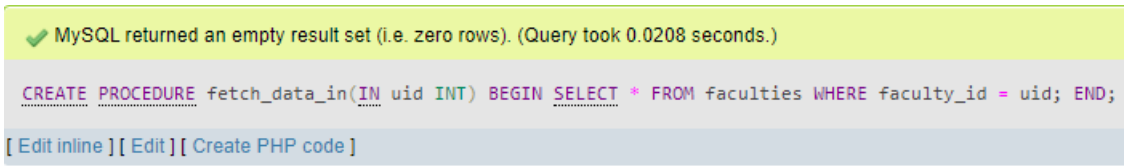
- SELECT * FROM table**;**
- INSERT INTO table VALUES (value)**;**

В прикладах оператори виконувані окремо тому в завершені запитів вказано стандартній розмежувач “ **;** **”**
Коли ми виконуємо в полідовності BEGIN – END тоді ми встановлюємо спец розмежувач “ // ”

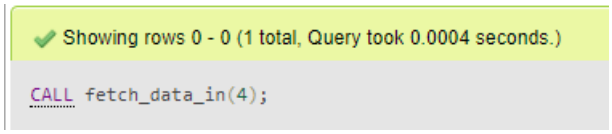
Параметр IN в procedure

Параметр **IN** встановлює значення variable, котре використовується в послідовності запитів BEGIN – END. Дозапишемо procedure, в котрій встановимо ідентифікатор факультета **IN** та зчитуємо дані факультета запитом **SELECT**

```
1 DELIMITER //
2
3 CREATE PROCEDURE fetch_data_in(IN uid INT)
4 BEGIN
5     SELECT * FROM faculties WHERE faculty_id = uid;
6 END //
7
8 DELIMITER ;
```



Тепер виконаємо procedure



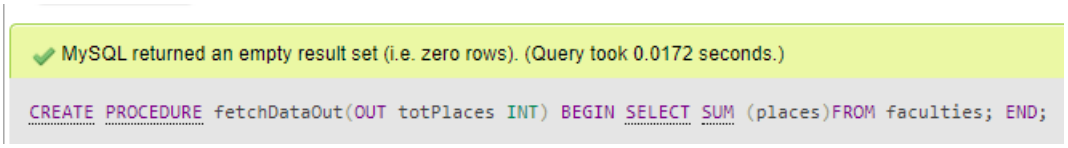
Тоді отримуємо:

faculty_id	name	places
4	Economy	50

Бачимо, що в нас зчитано екземпляр даних факультета по введеному ідентифікатору.

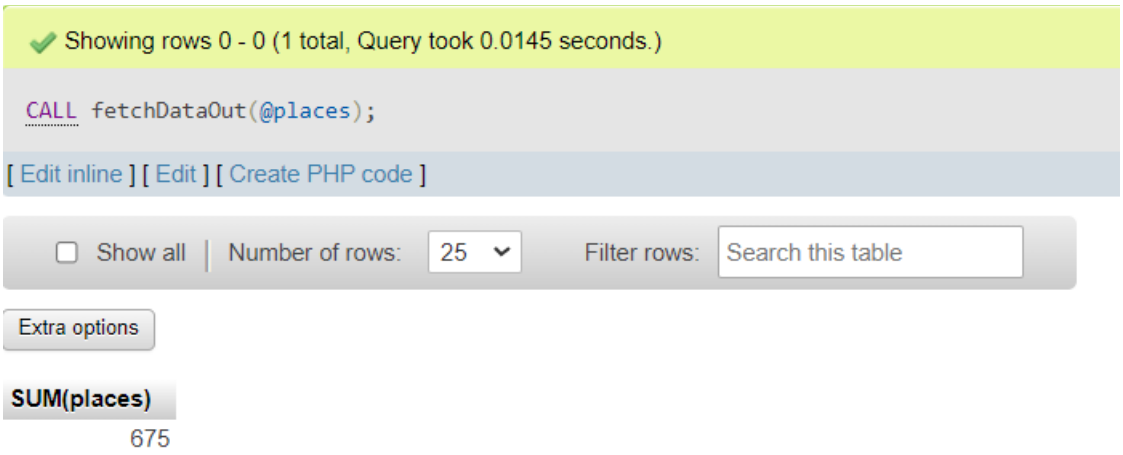
Параметр OUT

Параметр **OUT** використовується для повертання обчисленого значення в сеанс. Дозапишемо procedure обчислювання загального totPlaces:

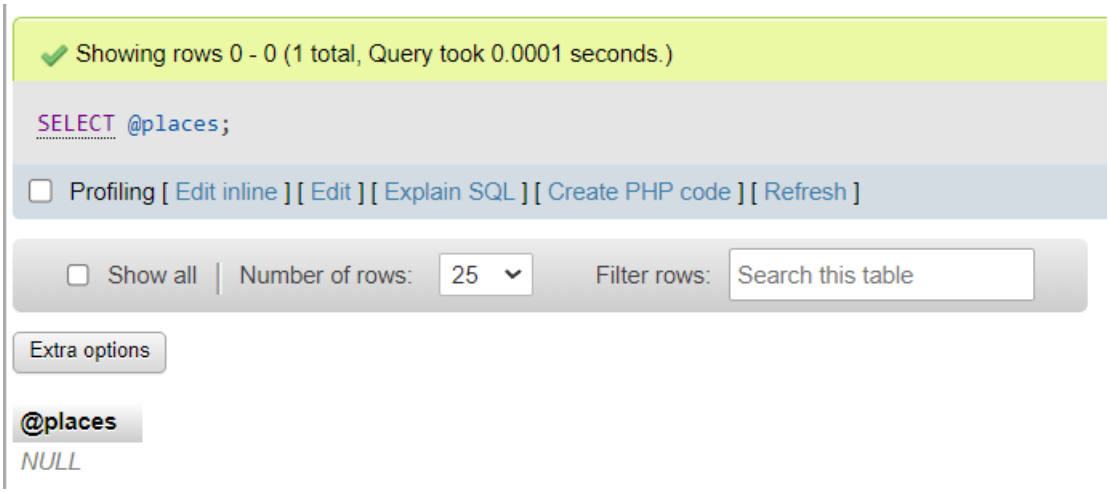


Зробимо виклик процедури використавши CALL

CALL fetchDataOut(@places)



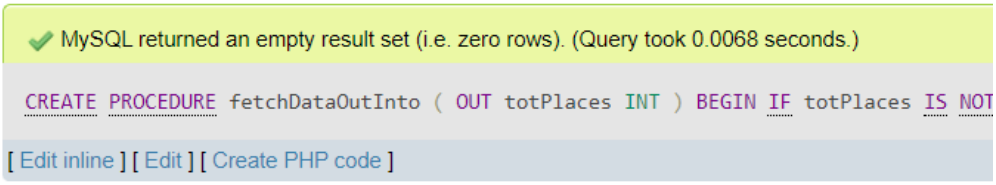
SELECT @places

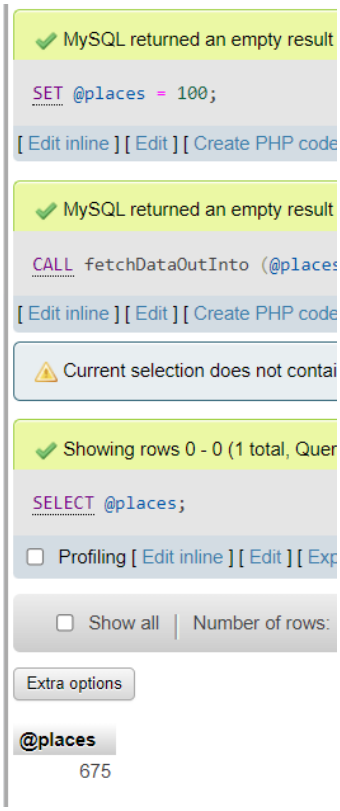


```
DELIMITER //

CREATE PROCEDURE fetchDataOutInto ( OUT totPlaces INT )
BEGIN
    IF totPlaces IS NOT NULL
    THEN
        SELECT totPlaces INTO totPlaces;
    ELSE
        SELECT SUM(places) INTO totPlaces FROM faculties;
    END IF;
END
//

DELIMITER ;
```





Оновлення локальних variables реалізує оператор **SELECT-INTO**

Параметр INOUT

Параметр INOUT поєднує параметри IN/OUT. Дозапишемо procedure в котрій встановимо ідентифікатор користувача разом з amount на котре виконується зменшування.

```
DELIMITER //

CREATE PROCEDURE update_data (IN uid INT, INOUT amount INT)
BEGIN
    DECLARE uPlaces INT;
    SELECT places INTO uPlaces FROM faculties;
  WHERE faculty_id = uid;
    IF amount > uPlaces THEN
      SIGNAL SQLSTATE '45000'
       SET MESSAGE_TEXT = "Insufficient places"
    ELSE
      UPDATE faculties
        SET places = uPlaces - amount
        WHERE faculty_id = uid;
    END IF;
    SELECT places into amount from faculties WHERE faculty_id = uid;
END //

DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0191 seconds.)

```
CREATE PROCEDURE update_data (IN uid INT, INOUT amount INT) BEGIN DECLARE
uPlaces INT; SELECT places INTO uPlaces FROM faculties WHERE faculty_id =
uid; IF amount > uPlaces THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
"Insufficient places"; ELSE UPDATE faculties SET places = uPlaces - amount
WHERE faculty_id = uid; END IF; SELECT places into amount from faculties
WHERE id = uid; END;
```

[Edit inline] [Edit] [Create PHP code]

```
SET @amount = 100;
CALL update_data (1,@amount);
SELECT @amount
```

Showing rows 0 - 0 (1 total, Query took 0.0002 seconds.)

```
SELECT @amount;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP cc

Show all | Number of rows: 25 Filter rows

Extra options

@amount
100

```
SET @amount = 1500;
CALL update_data (1,@amount);
```

Error

SQL query: Copy

```
CALL update_data (1,@amount);
```

MySQL said:

#1644 - Insufficient places

Пункт 1, практичного завдання

Реалізуйте procedure orders_summary(IN var1 TYPE1, OUT var2 TYPE2, INOUT var3 TYPE3);

```
DELIMITER //

CREATE PROCEDURE facult_change(IN facult_id INT, OUT names VARCHAR(100), INOUT min_amount INT)
BEGIN
  DECLARE faculty_place INT;

  SELECT places, name INTO faculty_place, names FROM faculties WHERE faculty_id = facult_id;

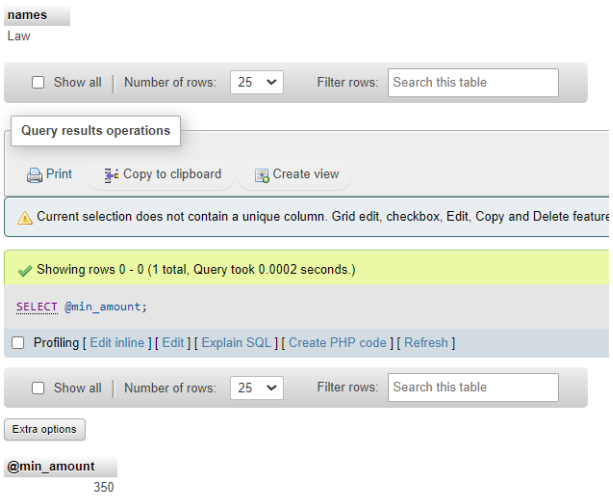
  IF min_amount > 1000 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = "Insufficient deposit";
  ELSE

    UPDATE faculties
    SET places = places + min_amount
    WHERE faculty_id = facult_id;
  END IF;

  SELECT names;
  SELECT places INTO min_amount FROM faculties WHERE faculty_id = facult_id;
END//

DELIMITER ;
```

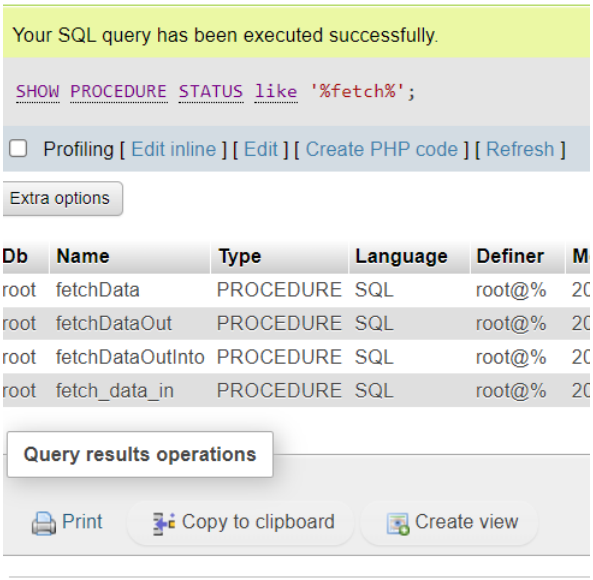
SET @faculty_id = 3;
SET @min_amount = 50;
CALL facult_change(@faculty_id, @places, @min_amount);
SELECT @min_amount



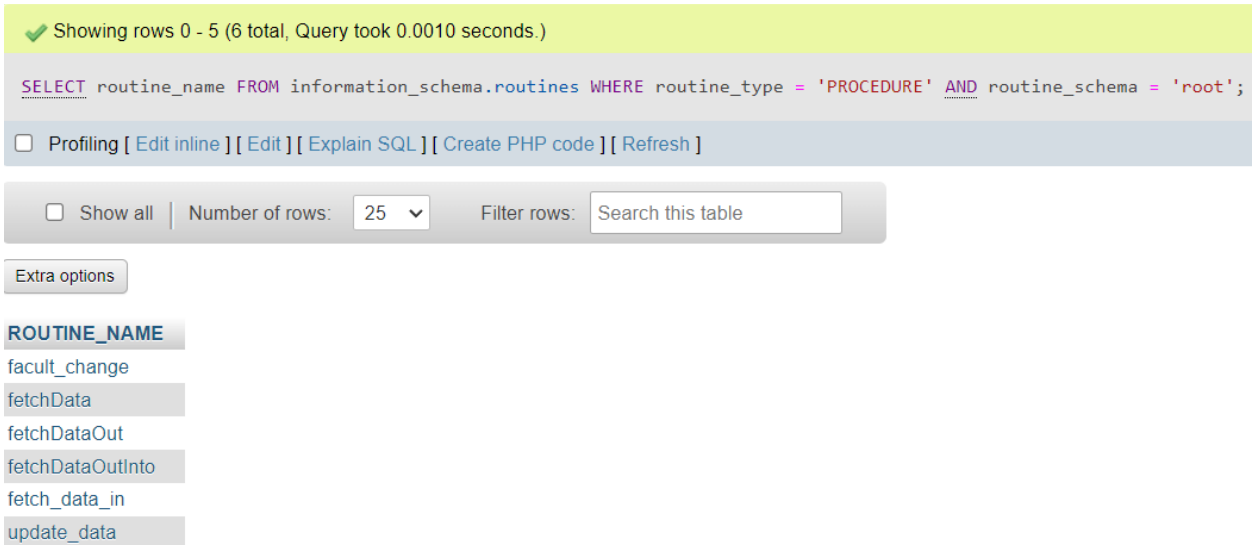
Зчитування procedures

Оператор SHOW PROCEDURE STATUS зчитує procedures.
SHOW PROCEDURE STATUS [LIKE 'pattern' | WHERE search_condition];
В запиті:

- SHOW PROCEDURE STATUS – оператор зчитування вмісту procedures;
- LIKE – зчитування procedures по шаблону найменувань;
- WHERE – зчитування procedures по умові



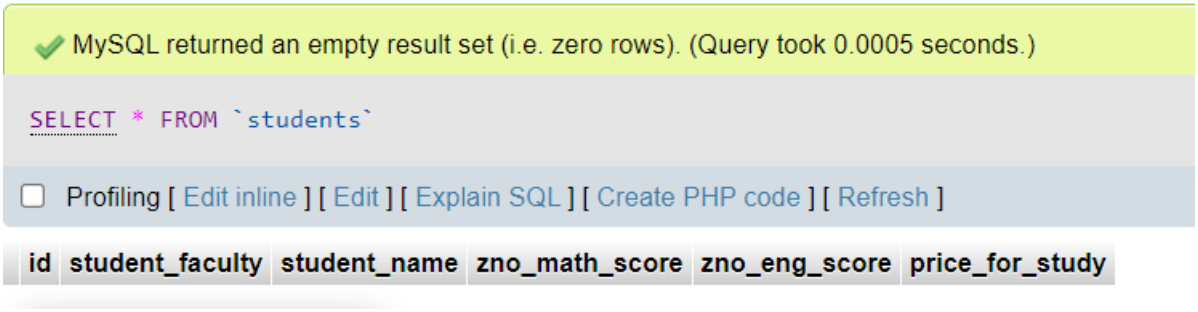
Зчитування найменувань procedures з сервера MySQL

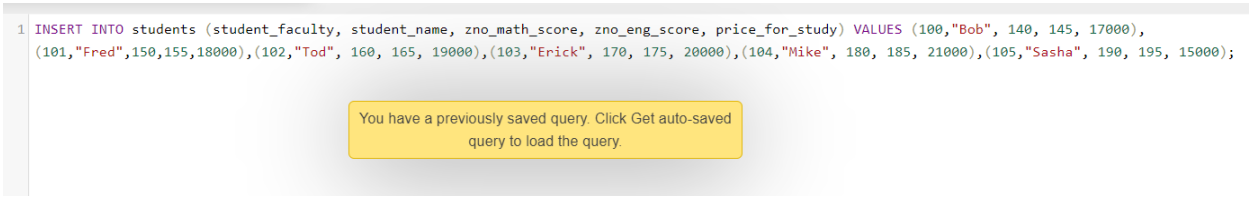


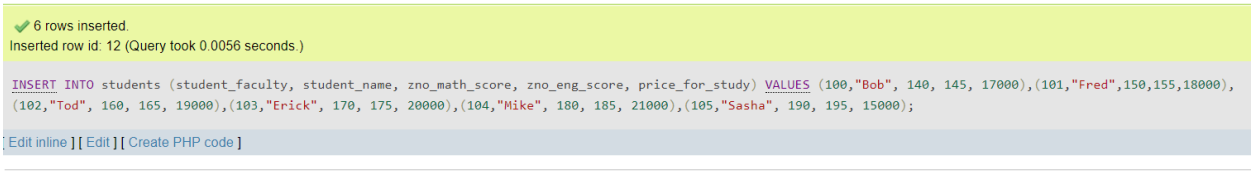
Тригер BEFORE UPDATE

Тригер включається під час виконування запиту оновлювання, але до оновлювання даних в таблиці. Оскільки тригер включається до запиту оновлювання, то може оновлювати нові значення до їх оновлювання в таблиці. Запит UPDATE використовуємо при оновлюванні таблиці. Можна використовувати послідовність запитів BEGIN-END щоб встановити кілька операторів у тригеріф

Побудуємо тестову таблицю та дозапишемо дані



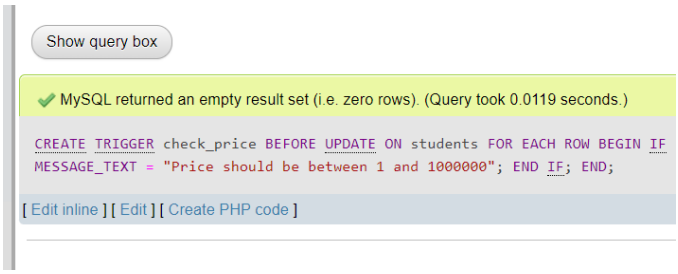





```
DELIMITER //

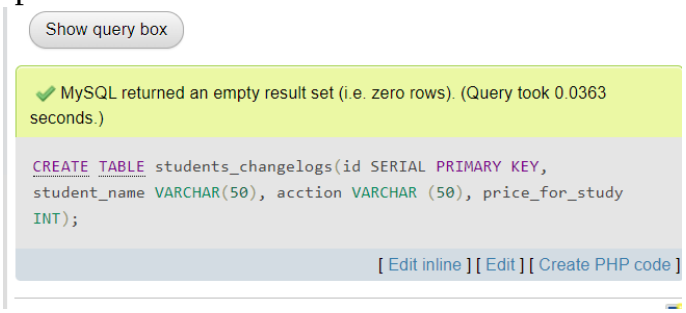
CREATE TRIGGER check_price
BEFORE UPDATE
ON students FOR EACH ROW
BEGIN
IF NEW.price_for_study < 1 OR NEW.price_for_study > 1000000
THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = "Price should be between 1 and 1000000";
END IF;
END//

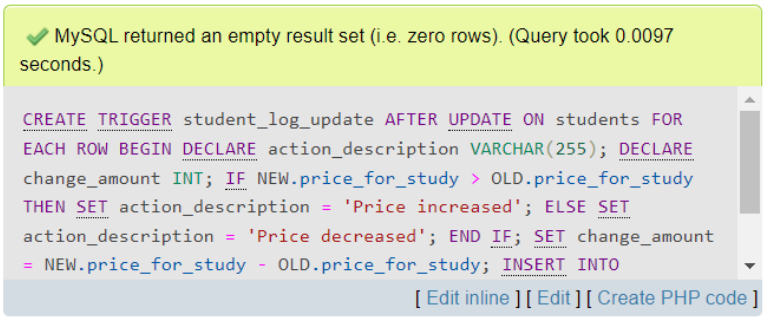
DELIMITER ;
```



Пункт 2 практичного завдання

Перевіряння тригера AFTER UPDATE





Проведемо тести:

```
1 UPDATE students SET price_for_study = 20000 WHERE id = 2;
```

id	student_name	acction	price_for_study
1	2	Price decreased	-8000
2	2	Price decreased	-9550
3	2	Price increased	19550

Тригер BEFORE DELETE

Тригер можна використовувати, щоб зберігати екземпляри даних, які потрібно вилучити, в іншій таблиці з метою архівування.

Для початку створемо архів відрахованих студентів.

Пропишемо наступну команду:
CREATE TABLE archive_students(id SERIAL PRIMARY KEY, student_faculty INT, student_name VARCHAR(100), zno_math_score INT, zno_english_score INT, price_for_study INT, deleted_at DATETIME DEFAULT NOW());

Тоді маємо:

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0326 seconds.)

```
CREATE TABLE archive_students(id SERIAL PRIMARY KEY, student_faculty INT, student_name VARC
deleted_at DATETIME DEFAULT NOW() );
```

[Edit inline] [Edit] [Create PHP code]

Встановлення тригера **before_delete**

```
CREATE TRIGGER before_delete
BEFORE DELETE ON students
FOR EACH ROW
BEGIN
    INSERT INTO archive_students(student_faculty, student_name, zno_math_score, zno_english_score, price_for_study, deleted_at)
    VALUES (old.student_faculty, old.student_name, old.zno_math_score, old.zno_eng_score, old.price_for_study);
END;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0200 seconds.)

```
CREATE TRIGGER before_delete BEFORE DELETE ON students FOR EACH ROW BEGIN INSERT INTO a
price_for_study, deleted_at) VALUES (old.student_faculty, old.student_name, old.zno_mat
```

[Edit inline] [Edit] [Create PHP code]

Встанолювання спільно виконуваних тригерів

Встановимо декілька тригерів з ідентичною умовою включання.

Включання нового тригера до існуючого встановлює умова PRECEDES; FOLLOWS встановлює включання тригера після існуючого

Встановимо тригери BEFORE UPDATE:

students_log_newprice – дозаписує нові і старі значення price в таблицю

students_price_logs щоразу, коли значення price_for_study оновлюється в таблиці students;

students_logs_triger – дозаписує інформацію користувачів в students_logs.

таблиця

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
SELECT * FROM `students_price_logs`
```

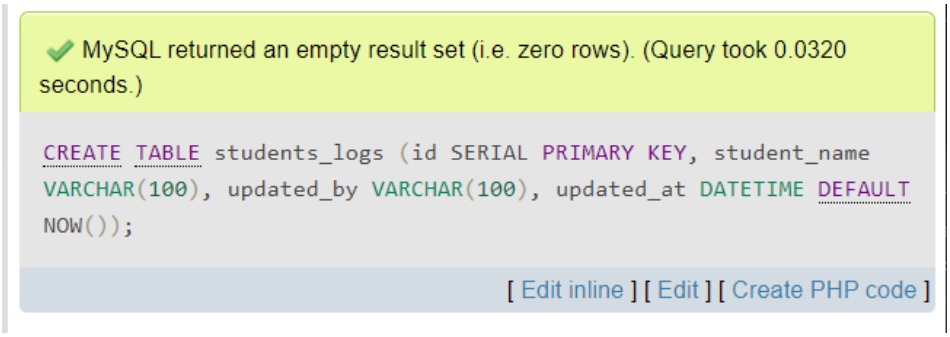
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

id	old_price	new_price	students_names	changed_at

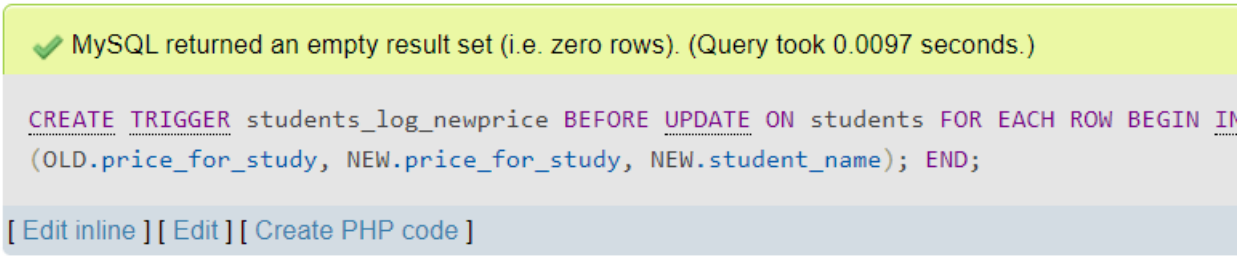
1 ший тригер

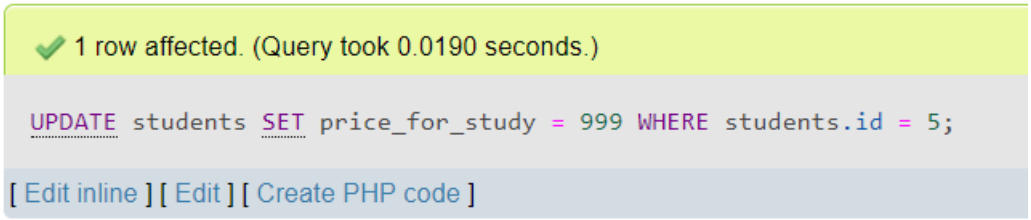
```
DELIMITER //
CREATE TRIGGER students_log_newprice
BEFORE UPDATE
ON students
FOR EACH ROW
BEGIN
  INSERT INTO students_price_logs (old_price, new_price, student_name)
  VALUES (OLD.price_for_study, NEW.price_for_study, NEW.student_name);
END //
DELIMITER ;
```

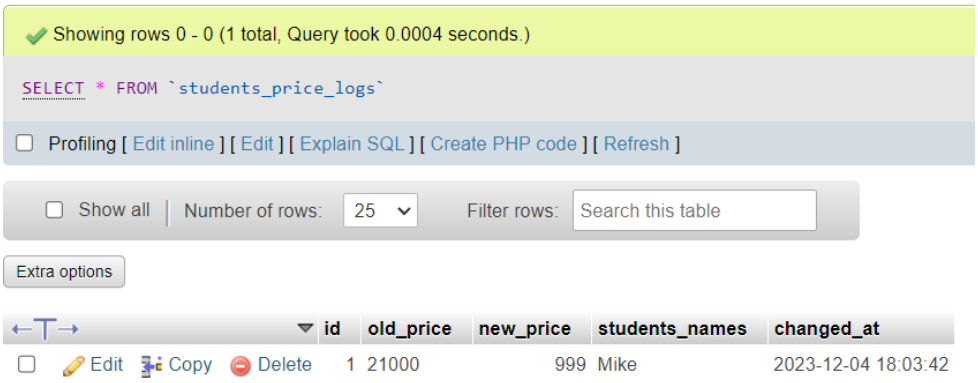
таблиця

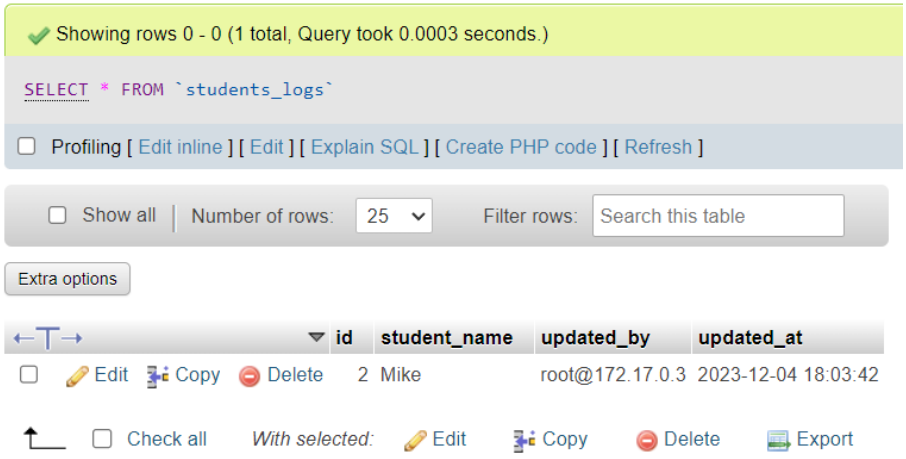


2тригер







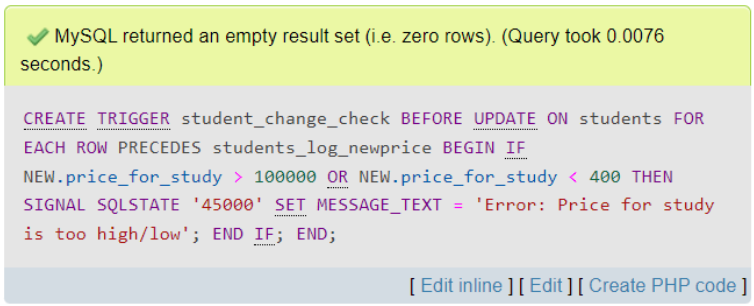


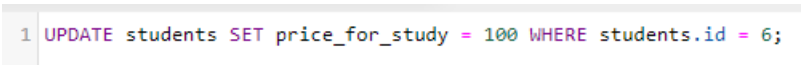
Виконуємо п3 практичного завдання

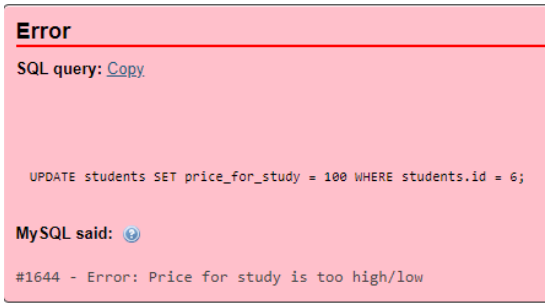
Встановіть тригер PREEDES, реалізуйте виконування тригерів

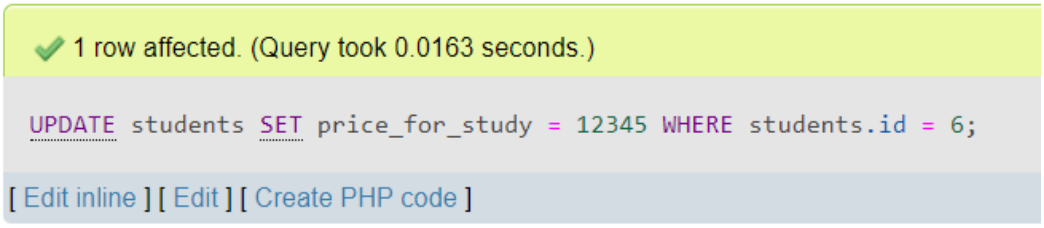
Встановимо тригер PRECEDES, реалізуємо виконування всіх тригерів. Створимо тригер, який буду перевіряти суму зміни кредиту перед тригерами занесення до таблиць students_logs (Follows students_price_logs), credits_summ_logs.

```
DELIMITER //
CREATE TRIGGER student_change_check
BEFORE UPDATE ON students FOR EACH ROW
PRECEDES students_log_newprice
BEGIN
  IF NEW.price_for_study > 100000 OR NEW.price_for_study < 400 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Price for study is too high/low';
    END IF
END //
DELIMITER ;
```









	id	old_price	new_price	students_names	changed_at
Edit Copy Delete	1	21000	999	Mike	2023-12-04 18:03:42
Edit Copy Delete	2	15000	12345	Sasha	2023-12-04 18:52:31

student_name	updated_by	updated_at
Mike	root@172.17.0.3	2023-12-04 18:03:42
Sasha	root@172.17.0.3	2023-12-04 18:52:31

Виконування procdure з тригера в MySQL

1. Встановимо procedure “withdraw”, що вираховує wamount від значення places до оновлювання екземпляра в faculties
2. Встановимо тригер BEFORE UPDATE. Щоразу, коли екземпляр оновлюється в procedure “withdraw”, тригер включається
3. Встановимо procedure “check_withdraw”, яка перевіряє при включенні тригера BEFORE UPDATE, чи доступний balance при виконуванні withdraw

```
DELIMITER //
CREATE PROCEDURE withdraw (IN uid INT, IN wamount INT)
BEGIN
 IF wamount <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Places should be >0';
  END IF;

  UPDATE faculties
  SET places = places - wamount
  WHERE id = uid;
END //
DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0081 seconds.)

```
CREATE PROCEDURE withdraw (IN uid INT, IN wamount INT) BEGIN IF
wamount <= 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
'Places should be >0'; END IF; UPDATE faculties SET places = places
- wamount WHERE id = uid; END;
```

[Edit inline] [Edit] [Create PHP code]

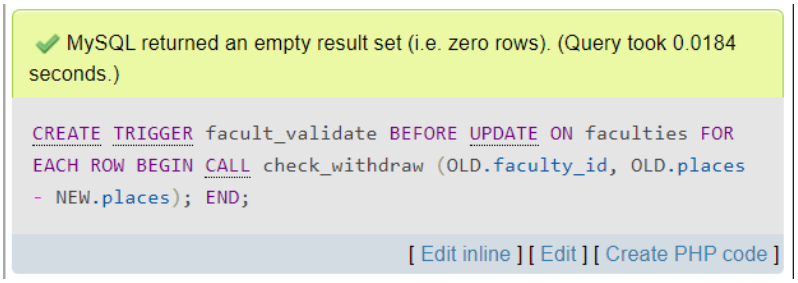
```
DELIMITER//
CREATE PROCEDURE check_withdraw(IN uid INT, IN wamount FLOAT)
BEGIN
 DECLARE nplaces INT;
  SELECT places INTO nplaces FROM faculties WHERE id = uid;
  IF wamount > nplaces THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Not enough places to withdraw fro faculty';
  END IF;
END //
DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0172 seconds.)

```
CREATE PROCEDURE check_withdraw(IN uid INT, IN wamount FLOAT) BEGIN
DECLARE nplaces INT; SELECT places INTO nplaces FROM faculties
WHERE id = uid; IF wamount > nplaces THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Not enough places to withdraw fro faculty'; END
IF; END;
```

[Edit inline] [Edit] [Create PHP code]

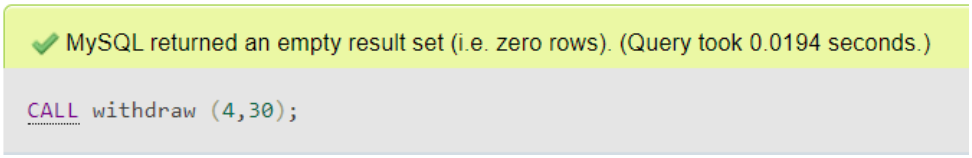
```
DELIMITER //
CREATE TRIGGER facult_validate
BEFORE UPDATE ON faculties FOR EACH ROW
BEGIN
 CALL check_withdraw (OLD.faculty_id, OLD.places - NEW.places);
END //
DELIMITER
```



faculty_id	name	places
1	Math	100

CALL check_withdraw(1,200);





faculty_id	name	places
4	Economy	20

Оператор LEAVE

Оператор використовується в послідовності запитів BEGIN-END, циклах

- WHILE,
- LOOP,
- REPEAT,

Умові IF-THEN-ELSE і виконує функцію подібну з оператором break мов програмування.

Створимо procedure leave_ex:

```
DELIMITER //
CREATE PROCEDURE leave_ex (IN x INT, IN y INT)
BEGIN
  DECLARE output VARCHAR(100) DEFAULT '';
    IF x > y THEN
      decr: LOOP
            SET output = CONCAT(output, x, ',');
            IF x = y THEN
            LEAVE decr;
           END IF;
           SET x = x -1;
         END LOOP;
     ELSE
       SET output = 'Both equal, cant iterate';
     END IF;
     SELECT output;
END//

DELIMITER ;
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0083 seconds.)

```
CREATE PROCEDURE leave_ex (IN x INT, IN y INT) BEGIN DECLARE output
VARCHAR(100) DEFAULT ''; IF x > y THEN decr: LOOP SET output =
CONCAT(outp, x, ','); IF x = y THEN LEAVE decr; END IF; SET x = x
-1; END LOOP; ELSE SET output = 'Both equal, cant iterate'; END IF;
END;
```

[Edit inline] [Edit] [Create PHP code]

Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)

```
CALL leave_ex(8,2);
```

[Edit inline] [Edit] [Create PHP code]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

output
8,7,6,5,4,3,2,

Пункт 4 практичного завдання

Реалізуйте PROCEDURE з умовою WHILE

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0094 seconds.)

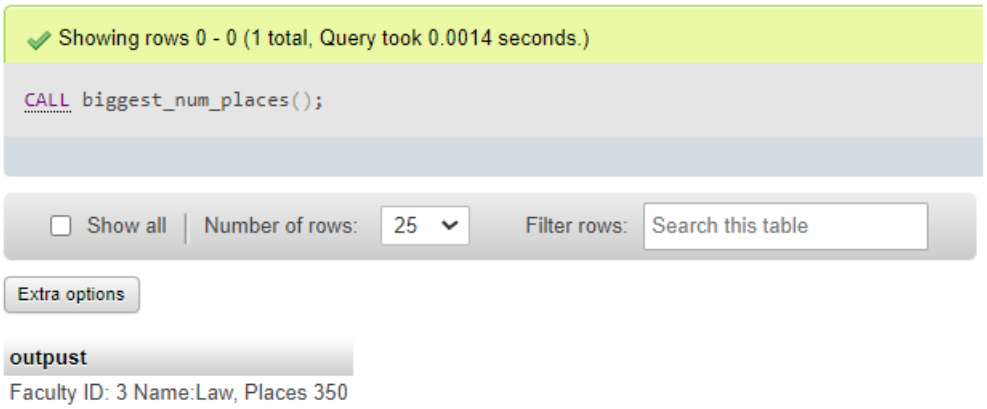
```
CREATE PROCEDURE biggest_num_places() BEGIN DECLARE counts INT DEFAULT 0; DECLARE num_places INT DEFAULT 0; DECLARE
max_places INT; DECLARE faculties_id INT; DECLARE places_summ INT; DECLARE faculty_name VARCHAR(100); DECLARE result
VARCHAR(100) DEFAULT ''; SELECT MAX(places) INTO max_places FROM faculties; SELECT COUNT(*) FROM faculties INTO
num_places; SET counts = 0; findloop: WHILE counts < num_places DO SELECT faculty_id, name, places INTO faculties_id,
faculty_name ,places_summ FROM faculties LIMIT 1 OFFSET counts; IF places_summ = max_places THEN SET result =
CONCAT(result, 'Faculty ID: ', faculties_id, ' Name:' , faculty_name ,', Places ', places_summ, '\n'); END IF; SET counts
```

[Edit inline] [Edit] [Create PHP code]

```
DELIMITER //
CREATE PROCEDURE biggest_num_places()
BEGIN
DECLARE counts INT DEFAULT 0;
DECLARE num_places INT DEFAULT 0;
DECLARE max_places INT;
```

```
DECLARE faculties_id INT;
DECLARE places_summ INT;
DECLARE faculty_name VARCHAR(100);
DECLARE result VARCHAR(100) DEFAULT '';

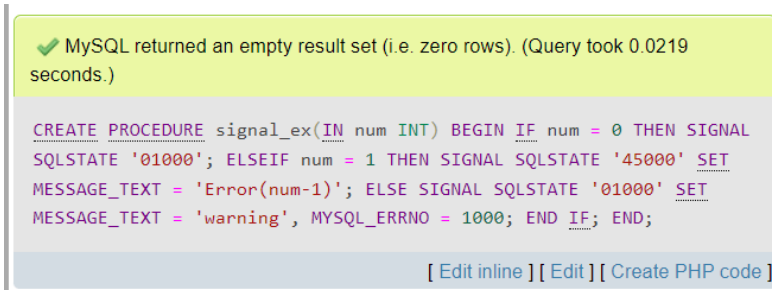
SELECT MAX(places) INTO max_places FROM faculties;
SELECT COUNT(*) FROM faculties INTO num_places;
SET counts = 0;
findloop: WHILE counts < num_places DO
SELECT faculty_id, name, places
INTO  faculties_id, faculty_name ,places_summ
FROM faculties
LIMIT 1 OFFSET counts;
IF places_summ = max_places THEN
SET result = CONCAT(result, 'Faculty ID: ', faculties_id, ' Name:' , faculty_name ,', Places ', places_summ, '\n');
END IF;
SET counts = counts + 1;
END WHILE;
SELECT result AS outpust;
END //
DELIMITER ;
```



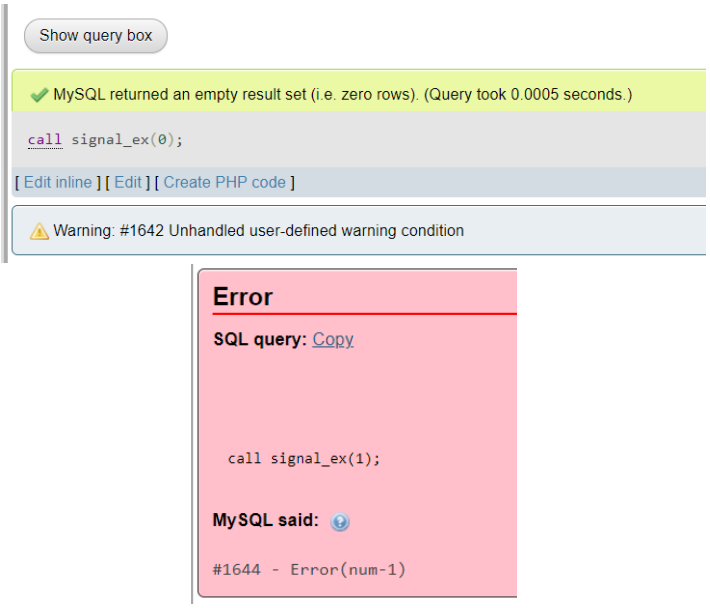
Оператор SIGNAL

Оператор використовується для повертання error і warnings в procedures, тригерах.

```
DELIMITER //
CREATE PROCEDURE signal_ex(IN num INT)
BEGIN
        IF num = 0 THEN
        SIGNAL SQLSTATE '01000';
  ELSEIF num = 1 THEN
        SIGNAL SQLSTATE '45000'
     SET MESSAGE_TEXT = 'Error(num-1)';
  ELSE
        SIGNAL SQLSTATE '01000'
     SET MESSAGE_TEXT = 'warning', MYSQL_ERRNO = 1000;
  END IF;
END //
DELIMITER;
```



Тестування:

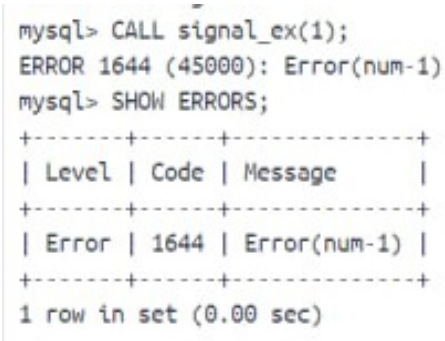


MySQL SHOW ERRORS

Оператор SHOW ERRORS повертає тільки errors, без warnings. Якщо потрібно повернути загальну кількість, можна використовувати агрегуючі функцію COUNT:

SHOW COUNT(*) ERRORS;

Умова повертання ERROR



Кількість хибних запитів

```
mysql> SHOW COUNT(*) ERRORS;
+-----------------------+
| @@session.error_count |
+-----------------------+
|                     1 |
+-----------------------+
1 row in set (0.00 sec)
```

MySQL SHOW WARNING

Діагностичний оператор SHOW WARNINGS подібний з SHOW ERRORS, але зчитує errors та warnings.

```
mysql> CALL signal_ex(3);
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> SHOW WARNINGS;
+---------+------+---------+
| Level   | Code | Message |
+---------+------+---------+
| Warning | 1000 | warning |
+---------+------+---------+
1 row in set (0.00 sec)
```

Зчитуємо warnings та errors використовуючи оператор SHOW WARNINGS:

```
mysql> CALL signal_ex(1);
ERROR 1644 (45000): Error(num-1)
mysql> SHOW ERRORS;
+-------+------+--------------+
| Level | Code | Message      |
+-------+------+--------------+
| Error | 1644 | Error(num-1) |
+-------+------+--------------+
1 row in set (0.00 sec)
```

Пункт 5 практичного завдання

Реалізуйте в PROCEDURE error і warnings

```
DELIMITER //
CREATE PROCEDURE error_and_warning(value INT)
BEGIN
  IF value = 0 THEN
     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error:Type 1';
   ELSEIF value > 0 AND 10000 < value THEN
     SIGNAL SQLSTATE '45000'  SET MESSAGE_TEXT = 'Error:Type 2';
   ELSEIF value > 0 AND 20000 < value THEN
     SIGNAL SQLSTATE '01000' SET MESSAGE_TEXT = 'Warning:Type 1';
   ELSEIF value < 0 THEN
     SIGNAL SQLSTATE '01000' SET MESSAGE_TEXT = 'Warning:Type 2';
   END IF;
END //
DELIMITER ;
```

Помилки

```
mysql> CALL error_and_warning(0);
ERROR 1644 (45000): Error:Type 1
mysql> SHOW WARNINGS;
+-------+------+--------------+
| Level | Code | Message      |
+-------+------+--------------+
| Error | 1644 | Error:Type 1 |
+-------+------+--------------+
1 row in set (0.01 sec)
```

```
mysql> CALL error_and_warning(15000);
ERROR 1644 (45000): Error:Type 2
mysql> SHOW WARNINGS;
+-------+------+--------------+
| Level | Code | Message      |
+-------+------+--------------+
| Error | 1644 | Error:Type 2 |
+-------+------+--------------+
1 row in set (0.00 sec)
```

Попередження

```
mysql> CALL error_and_warning(-1);
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> SHOW WARNINGS;
+---------+------+----------------+
| Level   | Code | Message        |
+---------+------+----------------+
| Warning | 1642 | Warning:Type 2 |
+---------+------+----------------+
1 row in set (0.00 sec)
```

Висновок.

У ході виконання лабораторної роботи №6, було проаналізовано можливості MySQL для бази даних Faculties. У веб-інтерфейсі були проведені CRUD операції для редагування даних.